

濠江中学
2025 学年
高三级毕业研究实践报告

主题: 互动式世界地图新闻展示网站

姓名: 謝宗霖

指导老师: 梁锦华

学科: 电脑

日期: 2026 年 4 月 30 日

目录

摘要 2

1 研究背景与意义 2

2 研究目标 2

2.1 新闻数据处理 2

2.2 新闻数据展示 3

3 开发准备 3

3.1 开发工具介绍 3

3.2 新闻数据抓取 3

3.2.1 开发语言 3

3.2.2 数据存储 3

3.2.3 新闻来源 4

3.3 新闻聚类 6

3.4 新闻解析 6

3.4.1 公共大模型 AI API 6

3.4.2 本地大模型 6

3.4.3 地理坐标转换 7

3.5 媒体声誉值 7

3.6 新闻热度计算 7

4 核心技术模块拆解 9

4.1 新闻数据处理全流程图 9

4.2 主调度模块 9

4.3 数据抓取模块.....10

4.4 新闻聚类模块.....11

4.5 新闻解析模块.....13

4.6 热度计算模块.....16

5 成果展示 16

6 遇到的挫折与解决办法 19

7 后续展望 19

8 总结 21

9 致谢 21

10 附录 23

摘要

本项目旨在解决当代中学生在搜集时事新闻时面临的“信息孤岛”与“视角偏差”问题。通过开发一个整合全球多源新闻的自动化处理系统，并将新闻在世界地图上可视化的网站，将碎片化的文字资讯转化为直观的地理空间分布，帮助用户（尤其是面临保送考试、需具备全球视野的学生）建立新闻事件与地理空间的深度联系，从而高效、多维地感知全球资讯。**关键字**：新闻地图；RSS 订阅；大模型调度；地理信息可视化

1 研究背景与意义

现在的资讯虽然多，但其实特别“碎”。我们要想看点不同地区的消息，得在好几个 App 之间来回切。特别是像我这样面临保送面试的学生，在搜集时事新闻时真的感到头疼：信息来源太散，东看一点西看一点，视角很容易变窄，尝试用 AI 又怕它出幻觉，最后不仅效率低，还容易陷入“信息孤岛”，对事情的看法产生偏差。

最让人纠结的是，传统的新闻 App 几乎都是清一色的“瀑布流”列表。我们每天刷到各种大新闻，但它们在地图上到底离我们多远？跟我们所在的城市有什么空间联系？这些信息在列表里是完全看不出来的。

所以我就想，能不能用技术手段把这些散落在全球的“点”连成“网”？这个项目的目标，就是把新闻从枯燥的“文字列表”变成直观的“动态地图”。我想打破那种孤立的信息状态，通过这种“地图式”的呈现方式，帮大家在看新闻时能瞬间建立起“事件与空间”的逻辑联系。这不仅是为了提高获取信息的效率，更是想通过这种全球视野，帮同学们培养起更具空间感的地理素养。

2 研究目标

本次研究的主题是“互动式世界地图新闻展示网站”，主要分为两个部分：

1. 新闻数据处理。
2. 新闻数据展示。

2.1 新闻数据处理

主要的目标是开发自动化的全球新闻聚合系统。此部分又分为：

(1) 新闻数据抓取

研究如何利用 Python 异步抓取 (Asyncio) 来自美联社 (AP)、法新社 (AFP) 等不同媒体的新闻资讯。

(2) 新闻聚类

从美联社 (AP)、法新社 (AFP) 等媒体同时抓取新闻时，关于“中东局势”的消息可能会有 10 条。如果地图上出现 10 个重叠的点，用户体验会很差。

研究目标：识别不同信源对同一事件的描述，将其合并为“一个事件点，多个新闻源”。

(3) 新闻解析

研究如何通过 LLM(大语言模型)将多个不同媒体的报道整合成一个报道，并提取新闻中的具体地点。

研究如何将地点转换成地理坐标于地图上展示。

(4) 新闻热度计算

研究如何确保地图能动态反映“当前正在发生”的热点，而非陈旧信息的堆砌。

2.2 新闻数据展示

研究如何将新闻资讯清晰地地图上展示给用户。

3 开发准备

3.1 开发工具介绍

在开发工具与平台方面，本项目尝试了两个集成开发环境 (IDE) 和 AI 辅助编程工具：

1. VS Code + Cline(AI 编程插件)

用 VS Code 搭配 Cline 插件，能直接让 AI 帮忙改代码、修 bug。项目里我用了它的免费模型，够用也方便。

2. Trae (AI 原生 IDE)

字节跳动的 AI 原生 IDE，从底层就把 AI 融进开发流程。我试了它的付费版的 GPT-5.3-codex 模型。

3. minimax-2.7 API

适合写代码、修 bug、重构，学生党友好。月费 29 元有 6000 次请求，性价比很高。

4. 网页端 AI 大模型

门槛低、免费、响应快。主要用来理思路、搭架构、做功能规划，写完代码后也用它做优化和检查。常用 Gemini、Grok、Deepseek、豆包等。

3.2 新闻数据抓取

3.2.1 开发语言

为了实现高效的数据处理与复杂的逻辑构建，本项目的新闻数据抓取部分选用 Python 作为核心开发语言。选用 Python 的首要原因是其拥有极其丰富的第三方生态系统，可拓展性十分强大。

3.2.2 数据存储

本项目采用 SQLite 作为数据存储方案，主要因为其轻量化、便捷性、适配性、开发效率等多方面考虑，非常适合大量新闻存储。

3.2.3 新闻来源

由于很多媒体网站的反爬功能非常强大，因此使用普通 Python 爬虫很难获取各网站的新闻。于是我另找办法，发现了一个古老但方便的技术——RSS (全称：Really Simple Syndication · 简易信息聚合)。

以 BBC 的 RSS 为例：

<https://feeds.bbc.co.uk/news/world/rss.xml> 为此媒体的 RSS 链接^[1]，在浏览器中打开会显示为 XML（可扩展标记语言，Extensible Markup Language）格式的代码。图 1 为其中一个 XML 新闻。

```
<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0" ?>
  <channel ?>
    <title>BBC News - World</title>
    <description>The latest news from around the world</description>
    <link>https://www.bbc.com/news/world</link>
    <pubDate>Wed, 22 Apr 2026 11:19:57 GMT</pubDate>
    <media:thumbnail width="240" height="135" url="https://ichef.bbci.co.uk/ace/standard/240/cpsprodpb/40de/live/992a3f20-3e29-11f1-b55d-0f258dce1735.jpg"/>
  </channel>
  <item ?>
    <title>EU approves €90bn loan for Ukraine as pipeline is turned on ending deadlock</title>
    <description>Ukraine says it has reopened the Druzhba pipeline, after months of stalemate over stalled oil supplies to neighbouring Hungary. </description>
    <link>https://www.bbc.com/news/articles/cnv8199r3yyo?at_medium=RSS&at_campaign=rss</link>
    <guid isPermaLink="false">https://www.bbc.com/news/articles/cnv8199r3yyo#0</guid>
    <pubDate>Wed, 22 Apr 2026 11:19:57 GMT</pubDate>
    <media:thumbnail width="240" height="135" url="https://ichef.bbci.co.uk/ace/standard/240/cpsprodpb/40de/live/992a3f20-3e29-11f1-b55d-0f258dce1735.jpg"/>
  </item>
</rss>
```

图 1: XML 格式示意图

这是标准的 RSS 2.0 新闻订阅格式里的一条新闻。主要内容有：

标签	作用
<title>	新闻标题：包含在 CDATA(字符数据) 块里的 HTML 格式文本。
<link>	新闻原文链接： https://www.bbc.com/news/articles...
<description>	新闻正文：包含在 CDATA(字符数据) 块里的 HTML 格式文本。
<pubDate>	发布时间：Wed, 22 Apr 2026 11:19:57 GMT
<media:thumbnail>	图片链接： https://ichef.bbci.co.uk/ace/standard/240...

(图片链接还可能在 <media_content> 里、在 <enclosures> 里、会以HTML的 格式藏在正文里。)

所以，获取了新闻网站的 RSS 链接，就获取了有标题、新闻原文链接、正文、图片链接、发布时间的完整新闻。^[2]

不过，上面只是说明了 BBC 用的是 RSS 2.0，而不同新闻平台可能会采用不同的订阅格式。除了 RSS 2.0 之外，常见的还包括 RSS 1.0、RSS 0.9x 以及 Atom 等格式，它们在标签命名和结构上略有差异，手动解析会十分繁琐且容易出错。

为了统一处理各类 RSS 订阅源，本项目选用了 Python 第三方库——feedparser。

feedparser 是一款专门用于解析 RSS、Atom 等 XML 格式新闻源的库，能够自动兼容多种订阅规范。它可以直接自动处理 XML 的 CDATA 文本、特殊字符、时间格式等常见问题，大幅简化数据提取流程。

不过，部分 RSS 源仅提供简短的新闻摘要，并不包含完整正文内容（如图 1），无法满足新闻数据详细分析的需求。

因此，系统需要根据每条新闻提供的原文链接，进一步访问新闻详情页，获取完整正文（如果网站可爬）。

为了高效、准确地从新闻网页中提取干净正文，本项目采用了 **trafilatura** 工具库。该库能够自动识别并剔除网页中的广告、导航栏、页脚、推荐列表等冗余内容，精准提取新闻主体文本，大幅提升正文获取的质量与效率。

通过 **feedparser** 解析 RSS 基础信息、**trafilatura** 提取完整正文，系统实现了高效、稳定、通用的新闻数据全自动采集。

如何获取 RSS 链接？

RSS 是一项诞生于二十世纪末的古老技术，时至今日，主流商业媒体已普遍弱化甚至关闭 RSS 订阅支持。究其原因，一方面是社交媒体、短视频平台等中心化封闭生态崛起，平台通过自有信息流直接聚合用户，不再需要开放的 RSS 协议；另一方面，RSS 无法提供用户行为数据、难以植入广告、严重分流官网流量，与媒体商业利益相悖，导致多数媒体主动放弃 RSS 支持。

尽管如今许多媒体不再原生提供 RSS 接口，但仍可通过第三方工具获取其结构化新闻源——**Folo+RSSHub**。



图 2: Folo+RSSHub 示意图

Folo^[4] 是一款开源的智能信息聚合工具，同时也是面向普通用户的现代化 RSS 阅读器与内容发现平台。Folo 提供了便捷的 RSS 检索与订阅管理功能，能够快速查找各类媒体的可用订阅源（如图 3）。

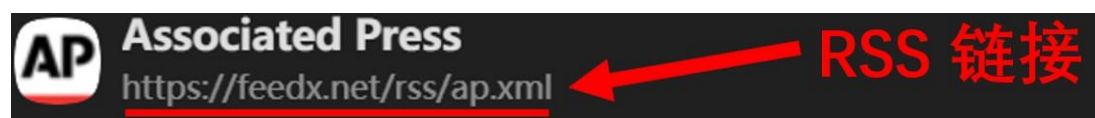


图 3: Folo 的 RSS 链接

RSSHub^[5] 是一个开源的订阅生成工具，支持为大量网站、社交平台、媒体机构生成标准化 RSS 链接，即使原网站未提供 RSS 也可正常获取新闻更新。

Folo 则提供了已生成的 RSS 链接，只需使用 RSSHub 公共实例，无需自行部署服务器，即可快速获取各类平台的新闻源。

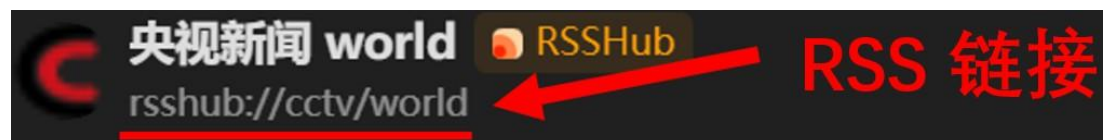


图 4: Folo 查找的基于 RSSHub 的 RSS 链接

地址	位置	维护者	在线状态
rsshub.rssforever.com	AE	Stille	

图 5: RSSHub 的其中一个公共实例^[6]

如图 4，在 Folo 里找到 CCTV 的 RSS(rsshub://cctv/world)，然后用 RSSHub 的公共实例（图 5）如 rsshub.rssforever.com 替换掉 RSS 的 `rsshub` 字段，变成一个可用的 RSS 链接：rsshub.rssforever.com/cctv/world，即可通过此链接获取 XML 格式的新闻。

3.3 新闻聚类

随着新闻数量不断增加，大量相似新闻会重复展示，影响用户体验。为解决这一问题，系统引入新闻聚类功能，将内容相似、主题相同的新闻自动合并为一组。

使用 `sentence-transformers` 的多语言模型将新闻文章转为向量，然后使用余弦相似度（Cosine Similarity）判断两条新闻内容是否相似。

3.4 新闻解析

在完成新闻聚类分组后，同类事件会汇集多条不同媒体的原始新闻素材。原始文本存在内容重复、角度不同的问题。

本项目接入在线和本地大模型 AI API，通过接口调用大模型能力，对聚类后的新闻组智能解析成一个简洁、中立的新闻。

3.4.1 公共大模型 AI API

本项目选用了公共免费大模型 API 来开发，无需本地部署，不被电脑配置限制。选用的公共免费大模型 API 厂商有：

- 1. 智谱 AI 开放平台^[13]：极速且处理中文语义极强，但容易触发敏感词限制。
- 2. 硅基流动 (SiliconFlow)^[14]：极速且能高吞吐的“算力中转站”，但只能体验普通模型。
- 3. NVIDIA NIM^[12]：能免费使用如 `deepseek-v3.2`、`minimax-m2.7` 等高端大模型，但需排队。

3.4.2 本地大模型

本项目使用了 Ollama 运行轻量大模型。
Ollama 是一款开源、轻量、跨平台的本地大语言模型（LLM）部署与管理工具，目的是让普通人也能在自己电脑上轻松跑起各种开源大模型，极简部署，不用复杂配置、不用高端显卡、不用懂深度学习底层技术。
本项目部署了通义千问 Qwen3.5:4b 轻量模型，因为它轻量化且实用性均衡，而且硬件适配门槛低，我的笔记本电脑也可稳定运行。

3.4.3 地理坐标转换

AI 提取出的事件地点为纯文字地名，无法直接在地图上展示。因此系统将地点文本传入Nominatim^[7]免费开源地理编码API，通过<https://nominatim.openstreetmap.org/search>公开接口，将文字地名转化为纬度、经度坐标。

3.5 媒体声誉值

媒体声誉值是作为热度计算、新闻数据择优的核心基础指标。
本系统使用 SCImago Media Rankings^[8] 提供的数字声誉指标（Digital Reputation Indicator, DRI）(图 6 的 Overall 值)。

Rank	Brand	Domain		Overall ↓	Content	Country	Language
1	The New York Times	nytimes.com	↗	89.00	General	United States	English +2
2	BBC	bbc.com	↗	87.00	General	United Kingdom	English +7
3	The Guardian	theguardian.com	↗	86.75	General	United Kingdom	English
4	Fox News	foxnews.com	↗	84.50	General	United States	English
5	Reuters	reuters.com	↗	84.00	General	United Kingdom	English

图 6: SCImago Media Rankings 排名示意图

SCImago 媒体排名保证透明且客观。
“我们利用四个指标——权威评分、引用域名、引用流量和信任流量——权重均为 25%，构建了一个反映媒体网站数字声誉的整体指标。结果确定了它们在排名中的相对位置，并允许比较数字发展与领导力。” (SCImago Media Rankings 官网对其排名的描述^[9])

3.6 新闻热度计算

Hacker News 是一家关于计算机黑客和创业公司的社会化新闻网站^[10]。Hacker News 的算法是少见的“公开、可解释”的排序方案，适合用于简单的项目。

这是 Hacker News 的热度算法^[11]：

$$\text{Score} = \frac{P - 1}{(T + 2)^{1.8}}$$

(1)

变量	描述
<i>Score</i>	热度分数
<i>P</i>	得票数
<i>T</i>	距离发帖的时间

- 决定 Hacker News 的帖子排名有三个因素：
- (1) 得票数 *P*：得票 (*P*) 越多，排名越高。
 - (2) 距离发帖的时间 *T*：越是新发表的帖子 (*T* 越小)，排名越高。

这种机制可以让用户优先看到最新鲜、最热门的帖子。

在 Hacker News 热度算法上改进的新闻热度算法，将原算法中的“投票数 P ”改为“媒体声誉值总和”：

$$\text{heat} = \frac{\sum \text{reputation}}{(T + 2)^{1.8}} \quad (2)$$

变量	描述
<i>heat</i>	新闻热度
<i>reputation</i>	媒体声誉值
<i>T</i>	距离最早新闻发布的时间

媒体声誉值总和：同一事件可能会被多个媒体报道，系统将所有报道来源的媒体声誉值相加，声誉值越高，热度越高。

新闻热度：用来给新闻做热度排名，热度高的新闻优先展示给用户。

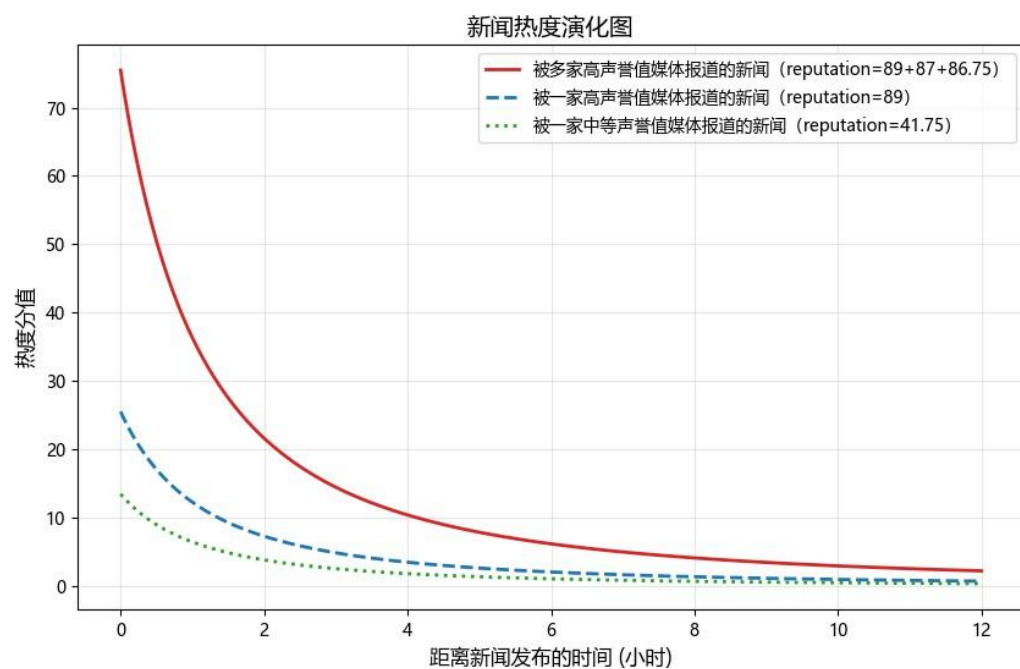


图 7: 新闻热度演化图

这个图可以看出在开始时红线（声誉值最高）热度最高，其次是蓝线，最后是绿线，它们随着时间流逝热度逐渐降低。这个改进的算法继承了原算法的机制，让用户优先看到最新鲜、最热门的新闻。

4 核心技术模块拆解

4.1 新闻数据处理全流程图

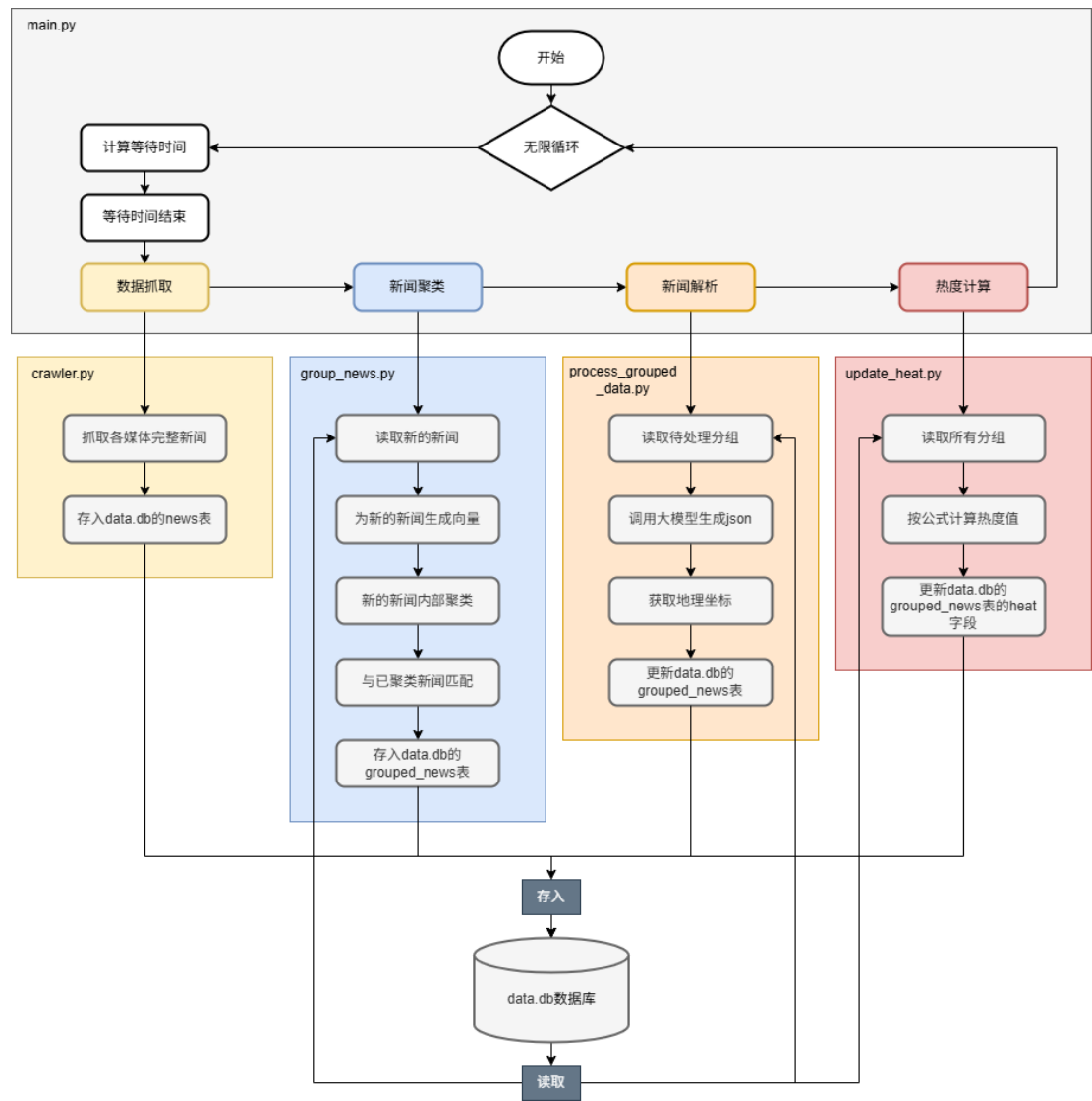


图 8: 新闻数据处理全流程图

4.2 主调度模块

主调度模块 `main.py` 是整个系统的总调度入口与定时任务管理器，负责串联爬虫、聚类、解析、热度计算四大核心模块，实现全自动、定时化的全流程数据处理。

4.3 数据抓取模块

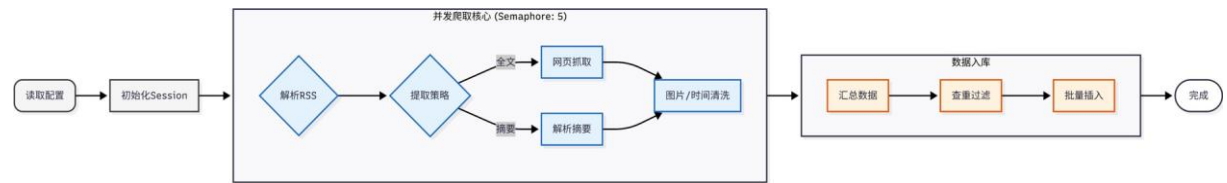


图 9: 数据抓取模块流程图

新闻抓取模块 `news_crawler.py` 是整个系统的数据底层，负责从各大主流媒体 RSS 订阅源自动采集原始新闻数据。

1. 配置代码分离设计

单独使用 `feeds.json` 配置文件统一管理所有 RSS 数据源。文件内可自定义媒体名称、RSS 链接、媒体信誉值、进入链接爬取正文开关、爬取开关。

```
{
  "source": "BBC",
  "rss_url": ["https://feeds.bbc.com/news/world/rss.xml"],
  "reputation": 87.00,
  "content_can_be_crawled": true,
  "crawl": true
},
```

图 10: `feeds.json` 里其中一个 RSS 配置

该设计实现配置与代码分离，后期新增、删减、停用新闻来源，调整媒体权重，仅需修改配置文件，无需改代码，提升系统扩展性与灵活性。

2. 功能介绍

- 异步批量拉取 RSS 源

程序启动后读取本地配置文件，异步加载所有启用的 RSS 地址，获取 XML。

- RSS 内容解析与基础信息提取

通过 `feedparser` 解析 XML 数据，批量获取单条新闻的基础信息，同时限制单个来源只保留最新 10 条新闻，避免数据冗余。

- 多策略图片提取

在 3.2.3 提到图片链接可能在 `<media:thumbnail>` 里、在 `<media_content>` 里、在 `<enclosures>` 里、会以 HTML 的 `<img>` 格式藏在正文里，因此代码把三种可能性都尝试一遍，保证能提取到图片链接（在新闻提供了图片的前提下）。

- 两种正文抓取模式

根据配置文件参数区分两种采集模式：

(1) "content_can_be_crawled": true, 即允许爬取原链接的媒体。这时会异步访问新闻原链接, 通过 `trafilatura` 清洗提取完整正文;

(2) "content_can_be_crawled": false, 即不允许爬取原链接的媒体。这时会直接截取 RSS 摘要作为正文内容。

- 时间标准化统一处理

由于不同国家媒体的时间格式、时区不统一, 程序对原始时间字符串转换为北京时间 YYYY-MM-DD HH:MM:SS 格式, 保障后续热度计算逻辑统一。

- 两层数据去重

以新闻唯一链接作为判断依据, 先在内存中剔除同批次重复数据, 再通过数据库查询比对历史数据, 自动忽略已存在新闻, 防止重复数据入库, 节约存储和防止影响后续热度计算。

- 数据存储

数据存储至 `data.db` 的 `news` 表。

新闻id	来源	声誉值	标题	原链接	发布时间	正文	图片链接
id	source	reputation	title	link	published	full_text	image_url
1	BBC	87	Former US	https://www	2026-04-2	Former US	https://ichef.bbci.co.uk
2	BBC	87	Russia scal	https://www	2026-04-2	Russia scal	https://ichef.bbci.co.uk
3	BBC	87	France urg	https://www	2026-04-2	France urg	https://ichef.bbci.co.uk
4	BBC	87	US Supren	https://www	2026-04-2	Supreme C	https://ichef.bbci.co.uk
5	BBC	87	France inv	https://www	2026-04-2	France inv	https://ichef.bbci.co.uk
6	BBC	87	Seven laws	https://www	2026-04-2	Seven laws	https://ichef.bbci.co.uk
7	BBC	87	Global fore	https://www	2026-04-2	Global fore	https://ichef.bbci.co.uk
8	BBC	87	Israeli 'dou	https://www	2026-04-2	Israeli 'dou	https://ichef.bbci.co.uk
9	BBC	87	Comey cha	https://www	2026-04-2	Comey cha	https://ichef.bbci.co.uk
10	BBC	87	Irish gover	https://www	2026-04-2	Irish gover	https://ichef.bbci.co.uk
11	AP	76	Police ider	https://apr	2026-04-2	Police ider	https://dims.apnews.cc
12	AP	76	Justice De	https://apr	2026-04-2	Justice De	https://dims.apnews.cc

图 11: news 表示意图

4.4 新闻聚类模块

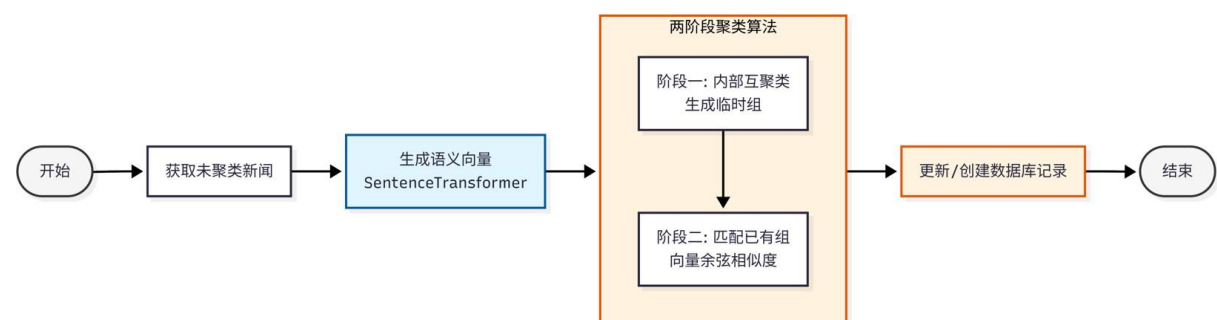


图 12: 新闻聚类模块流程图

新闻聚类模块 `group_news.py` 是把内容讲同一件事的多篇新闻，自动合并成一个“事件组”的核心模块。

功能介绍

• 两阶段聚类

阶段一：新新闻内部自聚类

对本次新增的新闻先进行一次聚类，生成临时分组。

阶段二：临时分组与历史分组匹配

将临时分组与数据库中已有事件分组比对，相似事件自动合并，不相似事件新建分组。

• 新闻文本向量化

使用 `SentenceTransformer` 预训练模型 `paraphrase-multilingual-MiniLM-L12-v2`。此模型支持中英双语新闻的语义理解，且模型体积小、推理速度快，适配普通笔记本硬件环境，无需高性能显卡。

将新闻标题与正文拼接后的文本转为向量，为相似度计算提供基础。

• 余弦相似度匹配

通过余弦相似度计算新闻向量间的语义相似度，数值越接近 1 表示内容越相似。本项目设定相似度阈值为 0.75。

• 数据存储

合并新闻 ID、来源 + 链接：自动合并同一事件的所有新闻 ID、来源和链接 (source,link,source,link...)。

取最早发布时间：保留事件的最早发布时间，保证事件时序的准确性。

图片择优：自动选取声誉值最高的新闻图片作为分组封面。

文本向量保存：分组中心向量以二进制 (BLOB) 格式存入数据库，每次运行时读取历史分组向量，无需重新计算。

分组id	原新闻id	英文标题	中文标题	英文全文	中文全文	最早发布时间	英文位置	中文位置	纬度	经度	种类	英文关键词	中文关键词	原新闻链接	原图片链接	向量	添加的新闻id	热度值
id	news_id	title_en	title_cn	full_text_en	full_text_cn	published	location_en	location_cn	latitude	longitude	category	keywords_en	keywords_cn	links	image_url	vector	added	heat
1	1					2026-04-29 10:01:50								BBC	https://bbc	https://unknown	\	0.3651
2	2	101				2026-04-29 21:01:57								BBC	https://bbc	https://unknown	\	2.4821
3	3	130				2026-04-29 22:18:03								BBC	https://bbc	https://unknown	\	3.1711
4	4					2026-04-29 23:05:55								BBC	https://bbc	https://unknown	\	2.1106
5	5					2026-04-29 23:10:55								BBC	https://bbc	https://unknown	\	2.1471
6	6	126				2026-04-29 21:50:37								BBC	https://bbc	https://unknown	\	3.0261
7	7					2026-04-29 21:26:08								BBC	https://bbc	https://unknown	\	1.4931
8	8					2026-04-29 22:55:15								BBC	https://bbc	https://unknown	\	2.0231
9	9	118				2026-04-29 08:24:05								BBC	https://bbc	https://unknown	\	0.6351
10	10					2026-04-29 20:48:45								BBC	https://bbc	https://unknown	\	1.3331
11	11					2026-04-25 02:53:02								AP	https://ap	https://unknown	\	0.0121
12	12					2026-04-25 02:23:02								AP	https://ap	https://unknown	\	0.0121
13	13					2026-04-25 02:23:02								AP	https://ap	https://unknown	\	0.0121
14	14					2026-04-25 01:23:02								AP	https://ap	https://unknown	\	0.0121
15	15					2026-04-25 00:23:01								AP	https://ap	https://unknown	\	0.0121
16	16					2026-04-24 23:23:03								AP	https://ap	https://unknown	\	0.0121
17	17					2026-04-24 22:53:02								AP	https://ap	https://unknown	\	0.0121
18	18					2026-04-24 22:53:02								AP	https://ap	https://unknown	\	0.0121
19	19					2026-04-24 22:53:02								AP	https://ap	https://unknown	\	0.0121
20	20					2026-04-24 22:53:02								AP	https://ap	https://unknown	\	0.0121

图 13: `grouped_news` 表示意图

4.5 新闻解析模块

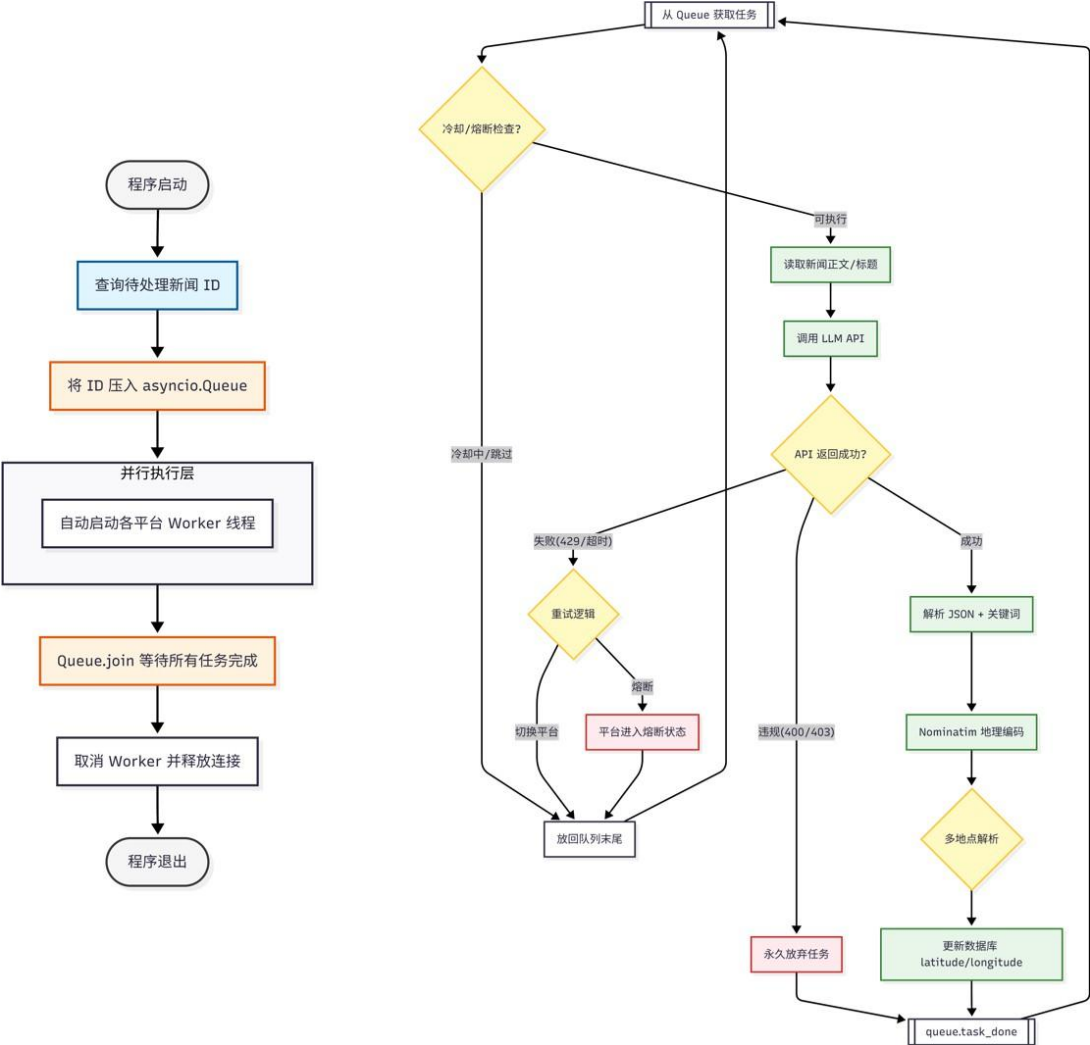


图 14: 新闻解析模块流程图

1. 配置代码分离设计

将大模型接口的地址、密钥、限流、并发、模型参数等全部抽离至独立的 `platforms.json` 配置文件，实现无侵入式扩展、灵活启停、多平台统一调度。

```
"silicon_Qwen3.5-4B": {
  "enabled": true,
  "url": "https://api.siliconflow.cn/v1/chat/completions",
  "rate_limit": 1.0,
  "concurrency": 2,
  "env_key": "SILICONFLOW_API_KEY",
  "model": "Qwen/Qwen3.5-4B",
  "extra_options": {
    "chat_template_kwargs": {
      "enable_thinking": false
    }
  }
},
```

图 15: platforms.json 里其中一个平台配置

2. 功能介绍

- 多平台大模型并发调度系统
- 由于免费公共大模型有众多限制（如并发数（concurrency）、每分钟请求数（RPM）、每分钟使用 Tokens 数（TPM）），因此需要一个保持稳定、安全的调度系统。

表 1: 多平台 AI 调度系统

机制	实现细节
并发设计	每个平台独配置并发数（concurrency），有并发数有多少，平台就有多少个 worker。worker 是系统的一个处理新闻的并发单体。这些 worker 会自动去任务池里领新闻，干完一个马上排队领下一个，全自动化，绝不偷懒。
限流设计	每个平台独配置限流时间，带随机抖动(配置时间 ±0.2 秒) 防封禁；如果返回 429，就是触发限流了，此时平台就等待 1 分钟。
熔断设计	如果连续 10 次失败，平台就要等待 15 分钟。
全局健康检测	所有平台全部熔断时自动停止，防止程序死循环。
跨平台失败转移	单条新闻在某个平台处理失败时，自动转移到其他可用平台重试，最多跨平台重试 2 轮，两轮后此新闻还失败，就跳过它。
失败冷却机制	同一新闻在同一平台失败后，120 秒冷却期内该平台不会再尝试处理这条新闻。防止一个平台死磕这条新闻。

提示词工程

调用 AI 的提示词明确定义了边界条件和反例，给出了正面和反面示例，包含严格的格式约束。

```

### 角色
你是一名专业的新闻编辑与GIS数据专家。任务：将新闻标准化为全球GIS格式。

### 任务流程
1. **摘要**：将新闻压缩为300-500字的流畅段落
2. **标题**：分别撰写专业的中英文标题
3. **提取地点**：按下方规则提取核心事件地点
4. **提取关键词**：3-6个中英文关键词
5. **分类**：从下方12个类别中**严格选择其一**

**类别（必须以英文原样输出，不可自创）**：
'politics', 'military', 'disaster', 'security', 'finance', 'diplomacy', 'society', 'tech', 'energy',
'environment', 'sports', 'entertainment'

---

### 地点提取规则（严格执行）

#### 一、什么是“核心事件地点”？
> 新闻中**主要行动/事件实际发生或集中所在**的具体地点。

**排除**以下情况（即使出现地点名词，也不提取）：
- 背景/来源地（如“中方在记者会上表示” → 不提取北京）
- 历史/背景地点（如“2020年伦敦发现...” → 不提取伦敦）
- 未来目的地（如“将于下周访问东京” → 不提取东京）
- 国籍/出生地（如“法国总统” → 不提取法国）

#### 二、提取粒度优先级（从细到粗）
地标 → 城市 → 省份 → 国家

| 情况 | 输出格式 | 示例 |
|-----|-----|-----|
| 提到具体地点（地标/城市/省份） | '具体地点, 国家' | '巴黎, 法国' |
| 仅提到国家（如宏观政策、全国性事件） | '国家' | '中国' |
| 独立地理实体（海峡/国际水域/跨境区域） | '实体名'（不加国家） | '霍尔木兹海峡' |
| 无任何核心地点可提取 | ""（空字符串） | "" |

#### 三、防冗余与别名
- ✗ '伦敦, 英国, 英国' → ✓ '伦敦, 英国'
- ✗ '美国, 美利坚合众国' → ✓ '美国'
- ✗ '新加坡, 新加坡' → ✓ '新加坡'
- ✓ 特别行政区：'香港, 中国'

#### 四、多地点处理
- 多个核心地点用 **; ** 分隔（分号+空格）
- 每个地点必须独立满足“核心事件”定义
- ✗ 错误：'法国'（遗漏了巴黎和里昂）
- ✓ 正确：'巴黎, 法国; 里昂, 法国'

---

### 输出格式（严格JSON）

返回**仅一个纯JSON对象**，无markdown代码块，无任何其他文字。

```json
{
 "title_en": "English title",
 "title_cn": "中文标题",
 "full_text_en": "English summary (300-500 words)",
 "full_text_cn": "中文摘要 (300-500字)",
 "location_en": "Paris, France; Lyon, France 或 """,
 "location_cn": "巴黎, 法国; 里昂, 法国 或 """,
 "location_num": 2,
 "keywords_en": ["keyword1", "keyword2"],
 "keywords_cn": ["关键词1", "关键词2"],
 "category": "类别"
}
```

---

### 语言要求
- 输入：中文或英文
- 输出：中英文均为**地道母语风格**，禁止机器翻译腔
- 中英文内容**独立撰写**，不是互译

---

### 输入新闻

来源标题："" + titles + ""

来源内容："" + contents + ""

```

图 16: 提示词示意图

新闻文章长度限制

如果把一个组别里的所有原新闻全部扔给AI，可能会导致处理时间变长或触发Tokens限制。为了限制文章长度，系统做了以下优化：

- (1) 只保留最有声誉的 2 篇报道。
- (2) 每篇报道只截取前 2000 字符。
- (3) 如该组别有新增新闻，则只用最有声誉的 2 篇新增新闻和组别已聚合新闻。

坐标获取算法

要搜索 [上葡京, 路氹城, 澳门] 的坐标，就会按以下顺序搜索。

- (1)[广州塔, 广州, 广东, 中国]（先搜最细的）
- (2)[广州, 广东, 中国]（搜不到就搜城市）
- (3)[广东, 中国]（搜不到就搜省份）
- (4)[中国]（最后保底搜国家）

这样就能保证有地点的新闻能在地图上正常显示。

处理后的新闻：

| 最早 | | | | | | | | | | | 英文 | 中文 | 原新闻 | 原图片 | 问量 | 添加的 | 热度值 | |
|------|---------|-------------|----------|--------------|--------------|--------------|------------|----------|-----------|-----------|-----------|-----------|--------------|------------|-----------------|-----------------|--------|--------|
| 分組id | 原新聞id | 英文标题 | 中文标题 | 英文全文 | 中文全文 | 发布时间 | 英文位置 | 中文位置 | 纬度 | 经度 | 种类 | 关键词 | 关键词 | 链接 | 链接 | 新聞id | added | heat |
| id | news_id | title_en | title_cn | full_text_en | full_text_cn | or published | location | location | clatitude | longitude | category | keywords | keywords | links | image url | vector | | |
| 1 | 1 | Former Off | 前高官斥五名前美 | Former Off | 前高官斥五名前美 | 2026-04-2 | Minab | Irar | 伊 | 27.150560 | 57.075281 | security | ["Pentagor | 五角大楼BBC | https://BBC | https://unknown | \ | 0.3635 |
| 2 | 2.101 | Russia to | 俄罗斯首 | Russia has | 俄罗斯首 | 2026-04-2 | Red Squan | 红场 | 俄罗 | 55.753590 | 37.621501 | military | ["Victory D | 胜利日BBC | https://BBC | https://unknown | \ | 2.4825 |
| 3 | 3.130 | Mali Escal | 马里局势 | Tensions ir | 萨赫勒地 | 2026-04-2 | Kidal | Mali | 基达尔 | 19.802089 | 0.7253125 | security | ["Mali", "T | 马里 | https://BBC | https://unknown | \ | 3.1715 |
| 4 | 4 | US Supren | 美国最高 | The US Sur | 美国最高 | 2026-04-2 | Louisiana | 路易斯安 | 第 | 30.870388 | -92.00712 | politics | ["Supreme | 最高法院BBC | https://BBC | https://unknown | \ | 2.1064 |
| 5 | 5 | France Lau | 法国调查 | French pro | 法国检察 | 2026-04-2 | Paris | Frank | 巴黎 | 48.858899 | 2.3200410 | security | ["France", | 法国 | https://BBC | https://unknown | \ | 2.1470 |
| 6 | 6.126 | Families S | 家属起诉 | Families of | 加拿大渥 | 2026-04-2 | Tumbler Ri | Tumbler | R | 55.126200 | -120.9980 | security | ["OpenAI", | OpenAI | https://BBC | https://unknown | \ | 3.0268 |
| 7 | 7 | Global For | 全球森林 | New satell | 最新卫星 | 2026-04-2 | Brazil | Colc | 巴西 | -10.33333 | -53.20000 | environme | ["Global Fc | 全球森林BBC | https://BBC | https://unknown | \ | 1.4936 |
| 8 | 8 | Israeli 'Do | 以色列双 | Lebanese | 黎巴嫩官 | 2026-04-2 | Majdal Zoi | 马尔达卢 | 第 | 33.150322 | 35.226028 | security | ["Double-t | 双击袭击BBC | https://BBC | https://unknown | \ | 2.0237 |
| 9 | 9.118 | Comey Sui | 科米因涉 | Former FBI | 前联邦调 | 2026-04-2 | Virginia | U | 弗吉尼亚 | 37.123224 | -78.49277 | politics | ["James Cc | 科米 | https://BBC | https://unknown | \ | 0.6355 |
| 10 | 10 | Irish Gov | 爱尔兰政 | Driven by | 在近期席 | 2026-04-2 | Dublin | Irel | 都柏林 | 53.349379 | -6.260559 | politics | ["Irish gov | 爱尔兰政BBC | https://BBC | https://unknown | \ | 1.3332 |
| 11 | 11 | Dead Body | 遗体在西 | Police in | 佛罗里达 | 2026-04-2 | Tampa Bay | 坦帕海湾 | 27.688641 | -82.57231 | society | ["Tampa B | 坦帕海湾AP | https://AP | https://unknown | \ | 0.0129 | |
| 12 | 12 | US Justice | 美国司法 | The U.S. D | 美国司法 | 2026-04-2 | | | | | | politics | ["capital p | 死刑 | https://AP | https://unknown | \ | 0.0128 |
| 13 | 13 | U.S. Sanct | 美国对中 | The U.S. g | 美国政府 | 2026-04-2 | Dalian | Chi | 大连 | 39.740278 | 122.25583 | politics | ["sanctions | 制裁 | https://AP | https://unknown | \ | 0.0128 |
| 14 | 14 | US Appeal | 美国上诉 | On Friday | 本周五 | 2026-04-2 | | | | | | politics | ["US Supre | 最高法院AP | https://AP | https://unknown | \ | 0.0126 |
| 15 | 15 | Wildfire in | 乔治亚州 | A devastat | 乔治亚州 | 2026-04-2 | NAHUNTA | 纳胡塔 | 乔 | 31.204390 | -81.98122 | environme | ["wildfire", | 野火 | https://AP | https://unknown | \ | 0.0124 |
| 16 | 16 | White Hou | 白宫记者 | The White | 白宫记者 | 2026-04-2 | Washingto | 华盛顿 | 美 | 38.895098 | -77.03638 | politics | ["White Hc | 白宫记者AP | https://AP | https://unknown | \ | 0.0123 |
| 17 | 17 | Rising Coll | 大学收入 | In a landm | 在职业业 | 2026-04-2 | | | | | | sports | ["NFL Draft | NFL选秀 | https://AP | https://unknown | \ | 0.0122 |
| 18 | 18 | FDA to Exp | FDA拟在 | The U.S. Fc | 美国食品 | 2026-04-2 | Enid | Okla | 俄克拉荷 | 36.396762 | -97.87913 | disaster | ["FDA revie | FDA审查 | https://AP | https://unknown | \ | 0.0122 |
| 19 | 19 | Devastatin | 致命风暴 | A fierce EF | 星期四夜 | 2026-04-2 | Enid | Okla | 俄克拉荷 | 36.396762 | -97.87913 | disaster | ["Tornado | 龙卷风 | https://AP | https://unknown | \ | 0.0122 |
| 20 | 20 | Connacir | 国会安 | The U.S. C | 美国国会 | 2026-04-2 | Washingto | 华盛顿 | 美 | 38.895098 | -77.03638 | politics | ["Connacir | 国会安AP | https://AP | https://unknown | \ | 0.0122 |

图 17: grouped_news 表示意图

4.6 热度计算模块

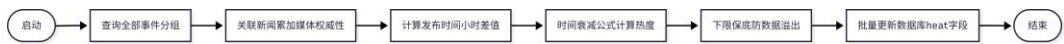


图 18: update_heat 流程图

5 成果展示

下面是网站的主页。

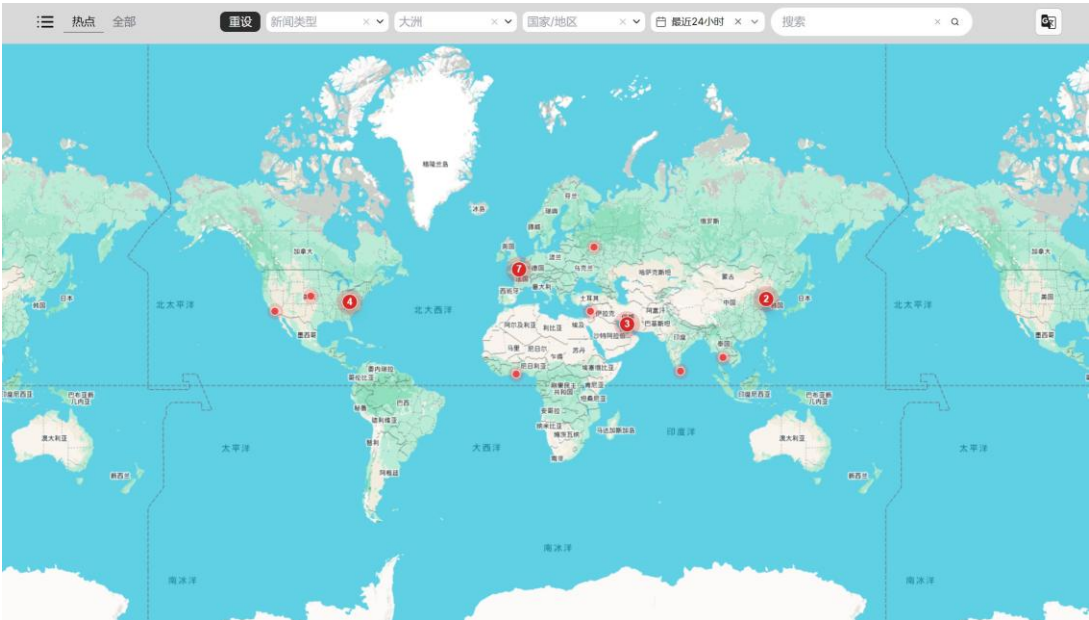


图 19: 网站主页

下面是网站主页顶部栏。



图 20: 网站主页顶部栏

顶部栏从左到右分别是：

- (1) **选择：热点/全部**。按热点后将会在地图上显示热度最高的前 20 条新闻；按全部后会显示全部新闻。
- (2) **新闻筛选**。能筛选新闻类型、大洲、国家/地区、时段。
- (3) **搜索框**。能搜索新闻中出现的字段。
- (4) **语言转换**。能切换中文/英文。

下面是地图上的新闻点。



图 21: 网站新闻点

点击新闻点后，右边会弹出新闻详情卡片。



图 22: 网站新闻点

新闻详情卡片会依次显示该新闻的标题、地点、发布时间、关键字、新闻类型、新闻图片和来源、正文、新闻链接和来源。



图 23: 网站新闻详情卡片

6 遇到的挫折与解决办法

1. 新闻网站反爬太强

最初使用普通的 `requests` 爬虫完全无法爬取法新社、路透社、美联社这些新闻网站，卡在了新闻数据获取这个最地基的一步。

解决办法

发现了 RSS 订阅这个功能，放弃直接爬取官网的方案，转用 RSS 订阅功能获取新闻数据。

2. 同一个平台的大模型死磕一条新闻

最初多平台大模型并发调度系统没有失败冷却机制，如果有新闻触发了 AI 平台的违禁词过滤，这时候系统就会报错。但是，由于当时其他 `Worker` 都在工作，剩下的那个工人就会盯着这条报错的新闻不放，一直处理失败->重新排队->又是它->再次尝试->再次失败... 短时间内在同一个平台上疯狂触发违禁词警告，这在 AI 平台看来就像是在恶意刷接口，我的账号面临极高的封禁风险。

解决办法

加入失败冷却机制，如果某个平台处理某条新闻失败了，系统会立刻给这条新闻打个“暂时标记”，在接下来的 120 秒内，这个平台绝对不会再碰这条新闻。

3. LLM 地理位置提取出错

初版提示词并没有写地点提取规则，导致提取出来的地点出现了 [中东, 美国]、[澳门, 澳门特别行政区] 这样的错误，还有一篇关于银河系的新闻提取了 [银河系] 这个不在地球的地点，这些问题导致无法在地图上正确显示其位置。

解决办法

在提示词里添加更严谨的地点提取规则，明确核心事件地点定义，指定地标、城市、省份、国家的顺序，无有效地点时返回空值，规范有多个地点时的格式与输出标准。通过精细化规则约束，虽然达不到完美，但大幅提升地点识别准确率，确保地理位置有效。

4. 获取地理坐标出错

免费的 `Nominatim` 在尝试获取太冷门的地点时很可能会失败，例如 `Nominatim` 搜索不到 [岭南中学, 澳门]，然后就直接输出个空的地点。

解决办法

优化获取地理坐标的算法，搜不到 [岭南中学, 澳门] 就搜下一级 [澳门]，这样虽然坐标不够精确，但能最大程度保证新闻在地图上能正常显示。

7 后续展望

目前的问题有：

1. 无 RSS 媒体无法采集

目前系统高度依赖标准的 RSS 协议。然而，部分媒体（如路透社）出于商业策略或平台限制，并未提供公开的聚合接口，也找不到第三方聚合接口

提升方案：

可接入联网检索型大模型，结合关键词定向全网检索技术，无需依赖 RSS 接口即可自动抓取网页正文、发布时间、媒体来源等标准化新闻字段，实现无 RSS 媒体的全自动采集，全面拓宽新闻来源覆盖面。

2. LLM 地理位置提取精度不足

现阶段依靠通用大模型提示词规则提取核心事件地点，面对跨境事件、多地点联动事件、模糊地名简称、历史背景混杂类新闻时，容易出现地点提取冗余、误提取背景地点、遗漏偏远小众点位、行政层级错乱等问题，直接影响后续地理编码与地图点位展示效果。

提升方案：

- 硬编码白名单：

建立一个本地敏感地点库。当新闻中包含这些关键词时，优先匹配你数据库中预设好的精准经纬度，而不是让 AI 去猜。

- RAG（检索增强生成）：

在发送给 LLM 之前，先检索一下本地地理库，把相关的背景信息发给它。例如：“注：新口岸位于澳门半岛，其坐标范围在...”。

3. Nominatim API 地理坐标获取精度不足

Nominatim 虽然免费且开源，但它对中文地名的理解（尤其是澳门这种中葡混杂的地名）确实不如高德（Amap）或 Google Maps。

提升方案：

- 使用高德/百度 API (针对中文环境)+Nominatim/Google Maps API(针对外语环境)。

4. 可拓展更多应用场景

- 智能时间线梳理功能：

依托聚类分组历史新闻数据，利用大模型时序语义排序能力，按新闻真实发布时序，自动筛选关键节点权威新闻，智能串联事件起因、发展、发酵、收尾全流程，直观呈现热点事件演变轨迹。

- 引入智能分析助手：

接入大模型，在前端构建对话式新闻聚合系统，允许用户直接输入模糊指令，如“对比一下过去一周东欧和中东地区的新闻热度分布。”

8 总结

经过一个月的设计与开发，本项目成功构建了一套从多源新闻自动采集、语义聚类、智能解析到热度计算的一体化新闻处理系统，并完成了能将新闻在地图上进行交互式可视化的网站。然而，对我个人而言，这个项目最大的收获并非技术成果本身，而是完整体验了软件工程从“0 到 1”的全过程。从初期因新闻网站反爬而屡屡碰壁，到转而研究 RSS 协议绕开壁垒；从脚本式代码难以维护，到学会配置分离、模块化设计与多平台 API 调度熔断；从追求“完美方案”到接受“权衡与迭代”——我在不断踩坑与填坑中，逐渐学会了如何像一名真正的软件工程师那样思考。这个项目是我踏上软件工程专业之路的第一块铺路石，它让我看到代码背后的力量。当然，目前系统仍存在许多不足，如无 RSS 媒体的采集难题、地理位置提取精度有待提升等，我会在未来的大学学习中继续完善这个项目，不断优化其稳定性和智能化水平。我深知这只是一个起点，但我已做好准备，迎接未来更广阔的技术挑战。

9 致谢

本项目的顺利完成，离不开许多人的支持与帮助。首先，衷心感谢我的指导老师梁锦华老师。从选题方向的确切到技术路线的把关，梁老师始终给予我耐心的指导和宝贵的建议，让我在遇到困难时能够坚定方向、持续推进。感谢学校提供的良好的学习环境和资源支持，让我有机会将课堂上学到的知识真正应用到实际项目中。感谢互联网上众多开源社区的贡献者，特别是 feedparser、trafilatura、sentence-transformers、Ollama 等开源工具的开发者们，你们的无私分享大大降低了项目的开发门槛。本系统仍存在诸多不足之处，许多技术方案也有待进一步优化，我深知自己目前的水平还很有限。但我会继续虚心学习、不断改进，努力不负老师和所有帮助过我的人的期望，在未来的软件工程学习中做出更有价值的作品。

参考文献

- [1] BBC News RSS. <https://feeds.bbc.co.uk/news/world/rss.xml> 3.2.3
- [2] RSS Advisory Board. <https://www.rssboard.org/rss-specification> 3.2.3
- [3] Folo. <https://app.folo.is/discover> 3.2.3
- [4] Folo. <https://app.folo.is/discover> 3.2.3
- [5] RSSHub. <https://docs.rsshub.app/> 3.2.3
- [6] RSSHub-公共实例. <https://docs.rsshub.app/zh/guide/instances> 5
- [7] Nominatim. <https://nominatim.org/> 3.4.3
- [8] SCImago Media Rankings. <https://www.scimagomedia.com/rankings.php> 3.5
- [9] SCImago Media Rankings-A trusted method. <https://www.scimagomedia.com/methodology.php> 3.5
- [10] wikipedia-Hacker News.
<https://zh.wikipedia.org/wiki/%E9%BB%91%E5%AE%A2%E6%96%B0%E9%97%BB>
3.6
- [11] 基于投票的热门计数算法策略. <https://www.cnblogs.com/binyue/p/6063353.html> 3.6
- [12] NVIDIA API Documentation. <https://docs.api.nvidia.com/> 3.4.1
- [13] 智谱 AI 开放文档. <https://docs.bigmodel.cn/cn/guide/start/quick-start> 3.4.1
- [14] 硅基流动开放文档. <https://docs.siliconflow.cn> 3.4.1

10 附录

代码链接: <https://github.com/Jackyxie008/News-web>